

Worksheet Answers: RLHF, Reasoning Models & Codex

CS 5090 · Software Engineering with Generative AI · Week 5 · answers with short explanations, for your own review before Midterm 1.
Where a question is open-ended, the answer below is one good answer, not the only one.

Part 1 · RLHF, PPO, DPO

1.1 Order the stages.

Pretraining → **SFT** → **RLHF**. Pretraining learns to predict the next token on raw text. SFT imitates human-written good responses. RLHF goes further: humans compare the model's own answers, and the model learns from a reward signal instead of a single correct answer.

1.2 InstructGPT Steps 2 and 3.

(a) Saying which of several responses is better is much easier for people than writing the ideal response, and comparisons carry more information than one demonstration.

(b) The softmax over the vocabulary is replaced by a single number: the **reward**. The model is trained so that higher-ranked responses get higher rewards.

1.3 True or false.

F: PPO does not use the preference pairs directly; it uses the reward model's score. **T**: that is the point of PPO, so training stays stable. **F**: DPO trains directly on the preferences, with no reward model and no PPO. **T**: that is the difference between SFT and RL.

Part 2 · Reasoning and test-time scaling

2.1 Fluent but wrong.

For example, a model explains clearly that "the loop checks every element from 0 to n" while the code has an off-by-one error, or gives a convincing explanation for code that calls a function that does not exist. A well-written explanation is not evidence that the code is correct.

2.2 Test-time or train-time?

Append "Wait" (budget forcing)	test-time
RLVR with GRPO	train-time
Sample many candidates and check them	test-time
SFT on reasoning traces	train-time

2.3 The s_1 plots.

Letting the model think longer raises accuracy, with the weights unchanged. The price is paid on every query: more tokens, so more cost and more latency.

Part 3 · RLVR and GRPO

3.1 Verifiable rewards.

Rewards **[1, 0, 1, 0]**. For code the check is automatic: run the tests. No human labeler and no learned reward model is needed, while "helpful" has no test you can run.

3.2 GRPO vs PPO.

GRPO eliminates the **critic (value) model**, which is usually as large as the LLM itself. Dropping it makes RL training much cheaper.

Part 4 · Codex, HumanEval, pass@k

4.1 A HumanEval problem.

A **function signature**, a **docstring** (the natural-language specification), and **hidden unit tests**. A solution is correct if it passes the tests.

4.2 $\text{pass}@1$ and $\text{pass}@3$.

$\text{pass}@1 = 2/4 = 50\%$ (P1, P4); $\text{pass}@3 = 3/4 = 75\%$ (P1, P3, P4). More samples at inference time solve more problems with

the same model: the model can often produce a correct solution even when its first attempt is wrong. That is spending compute at test time.

4.3 *Before merging AI-generated code.*

Run the tests (and add tests for the cases that matter), read and understand every line, check that every function and variable it uses actually exists, and review it for security problems. Verify with executed evidence, as in the Week 3 golden rules.